Patrones de diseño aplicados

Sistema de Préstamos Interbibliotecarios

TP Integrador 2026 · Tema 1: Biblioteca · Grupo 93
Desarrollo de Aplicaciones con Objetos · UTN – Facultad Regional Córdoba

Noviembre de 2026

1. Introducción

La consigna exige aplicar **al menos dos patrones de diseño**. El sistema aplica **seis** (State se usa dos veces): cada uno resuelve un problema concreto del dominio de la biblioteca, no se agregó ninguno «para cumplir». Este documento explica, para cada patrón, qué problema resuelve en el TP, cómo se implementó y por qué se eligió.

Cada patrón se presenta con el esquema de la cátedra: **problema**, **solución** (qué clase cumple cada rol del patrón), **implementación** (extractos reales del código con archivo y número de línea), **justificación**, **consecuencias** y **verificación** (los tests automáticos que lo comprueban).

Resumen

Patrón	Tipo	Clases	Problema que resuelve
State	Comportamiento	Remito, EstadoRemito, Pendiente, Empaquetado, Enviado, Entregado	Qué operación logística admite un remito según la etapa del envío (ida y retorno).
State	Comportamiento	Reserva, EstadoReserva y sus cuatro estados	Ciclo de una reserva: confirmar, cumplir o cancelar según corresponda.
Iterator	Comportamiento	Catalogo, Iterador, IteradorEjemplares, IteradorEjemplaresDisponibles	Recorrer el catálogo completo o solo lo disponible de una sede, sin exponer la colección.
Strategy	Comportamiento	EstrategiaMulta, SinMulta, MultaPorDia, MultaConTope	Política de multas por atraso intercambiable sin modificar Prestamo.
Singleton	Creacional	Configuracion	Plazos de préstamo y política de multas únicos para todo el sistema.
Factory	Creacional	FabricaPrestamos, Prestamo, PrestamoLocal, PrestamoInterbibliotecario	Crear el tipo de préstamo correcto y su vencimiento sin que el cliente conozca las subclases.
Observer	Comportamiento	Sujeto, Observador, Remito, NotificadorSocio	Avisar al socio en cada cambio de estado de su envío sin acoplar el remito al canal de aviso.

Dónde están los patrones

El sistema está organizado en capas: el **dominio** (`biblioteca/modelos/`), los **casos de uso** (`biblioteca/servicios.py`), la **persistencia** en SQLite (`biblioteca/persistencia/`), los **reportes** (`biblioteca/reportes.py`) y la **interfaz web** con Flask (`web/`). Todos los patrones viven en el dominio y no dependen de Flask ni de SQLite: por eso se prueban con tests unitarios sin base de datos. Los casos de uso los combinan, y la interfaz web nunca crea clases concretas del dominio por su cuenta.

2. State — Ciclo de vida del Remito

Es el patrón central del TP: la consigna indica que «el ciclo de vida del remito y sus estados será evaluado críticamente».

Problema

Un remito de un préstamo interbibliotecario pasa por **Pendiente → Empaquetado → Enviado → Entregado**. En cada etapa hay una sola operación válida, y cada transición tiene efectos propios: registrar su fecha, sacar los ejemplares de la sede (en tránsito no tienen ubicación), validar que ambas sedes sigan activas antes de despachar, ubicarlos en la sede destino al recibirlos y dejar la trazabilidad en el historial.

Sin el patrón, `Remito` tendría un atributo de texto `estado` y cada uno de sus tres métodos un *if/elif* sobre los cuatro estados: doce combinaciones repartidas en tres métodos. Agregar o cambiar un estado obligaría a revisarlos todos, y es fácil olvidar un caso y permitir, por ejemplo, recibir algo que nunca se despachó.

Solución

Rol del patrón	Clase	Responsabilidad
Contexto	Remito (remito.py)	Guarda el estado actual en _estado y delega empaquetar(), despachar() y recibir(). Expone cambiar_estado(), que registra el historial y notifica.
Estado (interfaz)	EstadoRemito (estados_remito.py)	Clase abstracta que declara las tres operaciones; _rechazar() centraliza el error.
Estados concretos	Pendiente, Empaquetado, Enviado, Entregado	Cada uno implementa la única operación que admite y rechaza las demás. Entregado rechaza todo: es el estado final.
Trazabilidad	HistorialRemito (remito.py)	Una entrada por estado: fecha y hora, nombre del estado, sede (None en tránsito) y observación.

Cualquier otra operación en cualquier estado → ReglaNegocioError (no cambia el estado ni deja rastro en el historial)

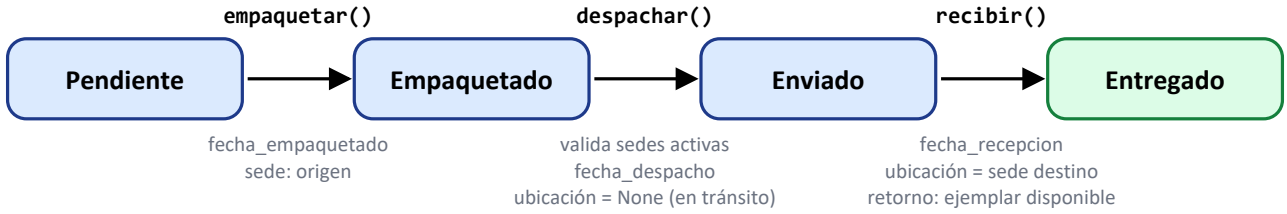


Figura 1. Transiciones del remito y sus efectos. Al crearse, el remito registra en el historial el estado inicial Pendiente.

Implementación

El contexto no decide nada: solo delega en el objeto estado. Todas las transiciones pasan por `cambiar_estado()`, que agrega la entrada de historial y notifica a los observadores (ver sección 8).

biblioteca/modelos/remito.py · líneas 58–73

```
58      def empaquetar(self, observacion=""):
59          self._estado.empaquetar(self, observacion)
60
61      def despachar(self, observacion=""):
62          self._estado.despachar(self, observacion)
63
64      def recibir(self, observacion=""):
65          self._estado.recibir(self, observacion)
66
67      def cambiar_estado(self, nuevo_estado, sede_actual, observacion=""):
68          self._estado = nuevo_estado
69          self.registrar_trazabilidad(nuevo_estado.nombre, sede_actual, observacion)
70          self.notificar()
71
72      def registrar_trazabilidad(self, estado, sede, observacion=""):
73          self.historial.append(HistorialRemito(estado, sede, observacion))
```

La interfaz del estado y el estado inicial. Pendiente solo acepta empaquetar(); las otras dos operaciones se rechazan con un mensaje para el usuario:

biblioteca/modelos/estados_remito.py · líneas 14–44

```python
class EstadoRemito(ABC):
    nombre = ""

    @abstractmethod
    def empaquetar(self, remito, observacion=""):
        pass

    @abstractmethod
    def despachar(self, remito, observacion=""):
        pass

    @abstractmethod
    def recibir(self, remito, observacion=""):
        pass

    def _rechazar(self, operacion):
        raise ReglaNegocioError(f"No se puede {operacion} un remito en estado {self.nombre}.")


class Pendiente(EstadoRemito):
    nombre = "Pendiente"

    def empaquetar(self, remito, observacion=""):
        remito.fecha_empaquetado = datetime.now()
        remito.cambiar_estado(Empaquetado(), remito.sede_origen, observacion)

    def despachar(self, remito, observacion=""):
        self._rechazar("despachar")

    def recibir(self, remito, observacion=""):
        self._rechazar("recibir")
```

Despachar y recibir concentran las reglas de negocio de cada etapa: al despachar se verifica que ninguna sede esté dada de baja y los ejemplares quedan en tránsito; al recibir se ubican en la sede destino y, si es el remito de retorno, vuelven a estar disponibles para prestar:

biblioteca/modelos/estados_remito.py · líneas 53–80

```python
    def despachar(self, remito, observacion=""):
        remito.prestamo.validar_sedes_activas()
        remito.fecha_despacho = datetime.now()
        for ejemplar in remito.prestamo.ejemplares():
            ejemplar.ubicacion_actual = None  # en tránsito
        remito.cambiar_estado(Enviado(), None, observacion)

    def recibir(self, remito, observacion=""):
        self._rechazar("recibir")


class Enviado(EstadoRemito):
    nombre = "Enviado"

    def empaquetar(self, remito, observacion=""):
        self._rechazar("empaquetar")

    def despachar(self, remito, observacion=""):
        self._rechazar("despachar")

    def recibir(self, remito, observacion=""):
        destino = remito.sede_destino
        remito.fecha_recepcion = datetime.now()
        for ejemplar in remito.prestamo.ejemplares():
            ejemplar.ubicacion_actual = destino
            if remito.es_retorno():
                ejemplar.marcar_disponible()  # volvió a su sede: se puede prestar otra vez
        remito.cambiar_estado(Entregado(), destino, observacion)
```

Remito de retorno

Cuando el socio devuelve todos los ejemplares en la sede destino, estos vuelven a su sede de pertenencia con un segundo remito (`tipo = Retorno`). **No hizo falta ningún estado nuevo**: el retorno recorre los mismos cuatro estados. Solo se agregó el tipo, y las propiedades `sede_origen` y `sede_destino` invierten el trayecto. Es una muestra concreta de lo que aporta el patrón: el ciclo se reutiliza sin tocar las reglas de transición.

El préstamo también consulta el estado del remito: la devolución solo se admite cuando el remito de ida ya está Entregado, y el retorno solo se genera cuando no quedan ejemplares pendientes:

`biblioteca/modelos/prestamo.py · líneas 144–157`

```
144     def validar_devolucion(self):
145         if self.remito is None or not isinstance(self.remito.estado, Entregado):
146             raise ReglaNegocioError(
147                 f"El material del préstamo {self.id_prestamo} todavía no llegó a la sede destino.")
148 
149     def generar_remito_retorno(self, numero_remito):
150         """Remito de vuelta (destino -> origen), una vez devueltos todos los ejemplares."""
151         if self.remito_retorno is not None:
152             raise ReglaNegocioError(f"El préstamo {self.id_prestamo} ya tiene un remito de retorno.")
153         if self.esta_activo():
154             raise ReglaNegocioError(f"El préstamo {self.id_prestamo} tiene ejemplares sin devolver.")
155         self.validar_sedes_activas()
156         self.remito_retorno = Remito(numero_remito, self, Remito.RETORNO)
157         return self.remito_retorno
```

Justificación

- **Cada regla está en un solo lugar.** Para saber qué pasa si se intenta despachar un remito pendiente basta con leer `Pendiente.despachar`.
- **No se pueden saltear etapas.** El remito no tiene un setter público del estado; la única forma de cambiarlo es que el estado actual llame a `cambiar_estado()`.
- **La trazabilidad no se puede olvidar (RN04).** Como toda transición pasa por `cambiar_estado()`, cada cambio deja su HistorialRemito con fecha, sede y observación.
- **Un intento inválido no deja rastro.** Los rechazos se lanzan antes de modificar nada: el historial y las fechas quedan intactos (lo verifica un test).
- **Alternativas descartadas.** Un *if/elif* sobre un texto (problema descripto arriba) o una tabla de transiciones permitidas: la tabla dice qué transición es válida, pero no resuelve que cada una tenga efectos distintos (fechas, ubicación, validación de sedes).

Consecuencias

- **Ventajas:** se puede agregar un estado (por ejemplo «Extraviado», del opcional de revisión de recepción) creando una clase, sin modificar las existentes; el ciclo de vida coincide uno a uno con el diagrama de clases.
- **Desventajas:** hay más clases, y cada estado repite los métodos que rechaza (se mitigó con `_rechazar()` en la clase base). Cada estado conoce a su sucesor; es aceptable porque el flujo es lineal y fijo.
- **Persistencia:** en la base se guarda el nombre del estado y el Repositorio reconstruye el objeto estado correspondiente (ver sección 10).

Verificación

`test_ciclo_completo_con_trazabilidad_y_ubicacion`, `test_transiciones_no_permitidas`, `test_no_se_despacha_hacia_sede_dada_de_baja`, la clase `TestRemitoRetorno` (`tests/test_modelos.py`) y `test_transicion_invalida_desde_la_base` (`tests/test_persistencia.py`).

3. State — Ciclo de vida de la Reserva

Problema

Una reserva nace **Pendiente**, se **confirma** y termina **cumplida** (el socio retiró el libro) o **cancelada**, ya sea desde Pendiente o desde Confirmada. Confirmar exige que el socio esté habilitado y la sede activa (RN14). A diferencia del remito, el ciclo **se bifurca** y tiene dos estados finales.

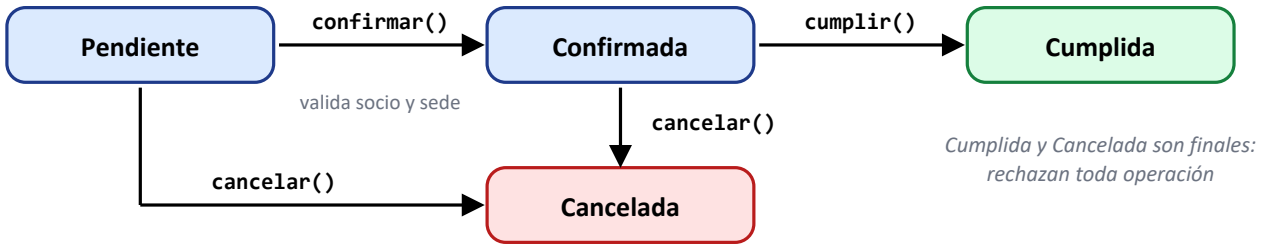


Figura 2. Estados de la reserva.

Solución

Rol del patrón	Clase	Responsabilidad
Contexto	Reserva (reserva.py)	Delega confirmar_reserva(), cancelar() y cumplir() en su estado.
Estado (interfaz)	EstadoReserva (estados_reserva.py)	Declara las tres operaciones.
Estados concretos	ReservaPendiente, ReservaConfirmada, ReservaCumplida, ReservaCancelada	Implementan las transiciones válidas; los dos estados finales rechazan todo.

Implementación

biblioteca/modelos/estados_reserva.py · líneas 32–56

```
32  class ReservaPendiente(EstadoReserva):
33      nombre = "Pendiente"
34
35      def confirmar(self, reserva):
36          reserva.validar_reserva(reserva.socio, reserva.sede)
37          reserva.cambiar_estado(ReservaConfirmada())
38
39      def cancelar(self, reserva):
40          reserva.cambiar_estado(ReservaCancelada())
41
42      def cumplir(self, reserva):
43          self._rechazar("cumplir")
44
45
46  class ReservaConfirmada(EstadoReserva):
47      nombre = "Confirmada"
48
49      def confirmar(self, reserva):
50          self._rechazar("confirmar")
51
52      def cancelar(self, reserva):
53          reserva.cambiar_estado(ReservaCancelada())
54
55      def cumplir(self, reserva):
56          reserva.cambiar_estado(ReservaCumplida())
```

biblioteca/modelos/reserva.py · líneas 32–39

```python
    def confirmar_reserva(self):
        self._estado.confirmar(self)

    def cancelar(self):
        self._estado.cancelar(self)

    def cumplir(self):
        """El socio retiró el libro reservado."""
```

Justificación

- **La validación está solo donde corresponde.** Únicamente `ReservaPendiente.confirmar` llama a `validar_reserva`: cancelar una reserva de un socio que luego fue inhabilitado sigue siendo posible, lo cual es correcto.
- **Un mismo mecanismo para todo el equipo.** Remito y Reserva resuelven su ciclo de vida igual, y la interfaz web solo necesita saber qué botones mostrar según el nombre del estado.
- **El patrón también sirve para ciclos no lineales:** la cancelación desde dos estados y los dos estados finales no requieren ninguna condición extra.

Consecuencias y verificación

Las mismas que en la sección 2: más clases a cambio de reglas localizadas y fáciles de extender. Tests: `test_ciclo_confirmar_y_cumplir`, `test_no_se_cumple_sin_confirmar`, `test_cancelar`, `test_confirmar_reserva`, `test_reserva_rechazada` y `test_reserva` (con la base de datos).

4. Iterator — Recorrido del Catálogo

Problema

El catálogo reúne los ejemplares de todas las sedes de la red. Hay que recorrerlo completo, y también solo los ejemplares **disponibles que pertenecen a una sede**: es la consulta de disponibilidad (proceso obligatorio 6) y la lista de la que se eligen los ejemplares al registrar un préstamo.

Si `Catalogo` expusiera su lista interna, cada cliente repetiría el filtro «disponible y de esta sede», y además podría agregar ejemplares directamente a la lista, salteando el control de códigos duplicados de `agregar_ejemplar()` (RN07).

Solución

Rol del patrón	Clase	Responsabilidad
Iterador (interfaz)	Iterador	Declara has_next() y next(), como en el diagrama de clases.
Iterador concreto	IteradorEjemplares	Recorre todos los ejemplares y guarda su propia posición.
Iterador concreto	IteradorEjemplaresDisponibles	Hereda del anterior y solo redefine has_next() para saltear los que no están disponibles o son de otra sede.
Agregado	Catalogo	Guarda la lista privada _ejemplares y crea los iteradores.
Cliente	servicios.consultar_disponibilidad	Usa el iterador sin conocer la lista.

Implementación

biblioteca/modelos/catalogo.py · líneas 8–54

```python
class Iterador(ABC):
    @abstractmethod
    def has_next(self):
        pass

    @abstractmethod
    def next(self):
        pass

    # Protocolo de Python, para poder usar el iterador en un for.
    def __iter__(self):
        return self

    def __next__(self):
        return self.next()


class IteradorEjemplares(Iterador):
    def __init__(self, ejemplares):
        self._ejemplares = ejemplares
        self._posicion = 0

    def has_next(self):
        return self._posicion < len(self._ejemplares)

    def next(self):
        if not self.has_next():
            raise StopIteration
        ejemplar = self._ejemplares[self._posicion]
        self._posicion += 1
        return ejemplar


class IteradorEjemplaresDisponibles(IteradorEjemplares):
    """Recorre solo los ejemplares disponibles que pertenecen a una sede."""

    def __init__(self, ejemplares, sede):
        super().__init__(ejemplares)
        self._sede = sede

    def has_next(self):
        while self._posicion < len(self._ejemplares):
            ejemplar = self._ejemplares[self._posicion]
            if ejemplar.disponible and ejemplar.pertenece_a_sede(self._sede):
                return True
            self._posicion += 1
        return False
```

biblioteca/modelos/catalogo.py · líneas 57–73

```python
class Catalogo:
    def __init__(self):
        self._ejemplares = []

    def agregar_ejemplar(self, ejemplar):
        if any(e.codigo == ejemplar.codigo for e in self._ejemplares):
            raise ReglaNegocioError(f"Ya existe un ejemplar con código {ejemplar.codigo}.")
        self._ejemplares.append(ejemplar)

    def crear_iterador(self):
        return IteradorEjemplares(self._ejemplares)

    def crear_iterador_disponibles(self, sede):
        return IteradorEjemplaresDisponibles(self._ejemplares, sede)

    def __iter__(self):
        return self.crear_iterador()
```

El cliente, la consulta de disponibilidad que usan la web y el formulario de préstamos:

biblioteca/servicios.py · líneas 85–88

```python
def consultar_disponibilidad(repo, id_sede, isbn=None):
    """Ejemplares disponibles de una sede, recorridos con el iterador del Catalogo."""
    catalogo = repo.catalogo(isbn or None)
    return list(catalogo.crear_iterador_disponibles(repo.sede(id_sede)))
```

Justificación

- **Encapsulamiento:** la colección es privada y la única forma de agregar es `agregar_ejemplar()`, que impide códigos repetidos.
- **El filtro está en un solo lugar.** «Disponible y perteneciente a la sede» se define una vez, en `IteradorEjemplaresDisponibles.has_next()`.
- **Recorridos independientes:** cada iterador tiene su propia posición, así que se pueden hacer varios recorridos simultáneos sin interferir.
- **Interfaz de la cátedra y de Python a la vez:** se respeta `has_next()` / `next()` con `StopIteration` al agotarse, y la interfaz además implementa `__iter__` y `__next__`, de modo que el iterador funciona en un `for` o con `list()`.

Consecuencias

- **Ventajas:** se pueden agregar recorridos nuevos (por ejemplo, por libro o por estado físico) con otra subclase, sin tocar Catalogo ni sus clientes.
- **Desventajas:** en Python las listas ya son iterables, así que el patrón agrega clases; se justifica por el filtrado y por ocultar la colección. El filtro se hace en memoria: con un catálogo muy grande convendría filtrar directamente en SQL.

Verificación

`test_recorre_todos_los_ejemplares_y_lanza_stop_iteration`, `test_se_puede_usar_en_for`, `test_rechaza_ejemplar_duplicado`, `test_solo_disponibles_de_la_sede` y `test_prestamo_local_y_disponibilidad`.

5. Strategy — Cálculo de multas

Problema

Controlar los vencimientos (RN03) implica cobrar multas por atraso, pero la consigna no fija la política y los valores actuales son provisorios. Las opciones razonables son varias: no cobrar (por ejemplo, durante un período de gracia), un monto por día y por ejemplar, o un monto por día con un tope. Con *if/elif* dentro de `Prestamo`, cada política nueva obligaría a modificar una de las clases centrales del dominio.

Solución

Rol del patrón	Clase	Responsabilidad
Estrategia (interfaz)	EstrategiaMulta	Declara calcular(dias_atraso).
Estrategias concretas	SinMulta, MultaPorDia, MultaConTope	Cada una implementa una política; los montos se configuran por constructor.
Contexto	Prestamo.calcular_multa(estrategia, fecha)	Calcula los días de atraso de cada ejemplar y delega el monto en la estrategia recibida.
Quién elige	Configuracion().estrategia_multa	La estrategia vigente se define en un único lugar (Singleton, sección 6).

Implementación

biblioteca/modelos/multas.py · líneas 9–36

```python
class EstrategiaMulta(ABC):
    @abstractmethod
    def calcular(self, dias_atraso):
        pass


class SinMulta(EstrategiaMulta):
    def calcular(self, dias_atraso):
        return 0


class MultaPorDia(EstrategiaMulta):
    def __init__(self, monto_diario):
        self.monto_diario = monto_diario

    def calcular(self, dias_atraso):
        return dias_atraso * self.monto_diario


class MultaConTope(EstrategiaMulta):
    """Monto diario, pero sin superar un máximo por ejemplar."""

    def __init__(self, monto_diario, tope):
        self.monto_diario = monto_diario
        self.tope = tope

    def calcular(self, dias_atraso):
        return min(dias_atraso * self.monto_diario, self.tope)
```

El contexto usa polimorfismo: invoca `estrategia.calcular(...)` sin preguntar de qué clase concreta se trata, igual que en la guía de la cátedra:

biblioteca/modelos/prestamo.py · líneas 108–113

```python
    def calcular_multa(self, estrategia, fecha=None):
        """Suma la multa de cada ejemplar. La estrategia la elige quien llama
        (normalmente Configuracion().estrategia_multa), no el préstamo."""
        fecha = fecha or date.today()
        return sum(estrategia.calcular(d.dias_atraso(self.fecha_vencimiento, fecha))
                   for d in self.detalles)
```

Un cliente, el reporte de préstamos vencidos y multas, toma la estrategia de la configuración:

biblioteca/reportes.py · líneas 28–33

```
28  def prestamos_vencidos_y_multas(repo, fecha=None):
29      """RR02: préstamos vencidos (activos) y multas acumuladas por socio, a `fecha`."""
30      fecha = fecha or date.today()
31      estrategia = Configuracion().estrategia_multa
32      vencidos, por_socio = [], {}
33      for p in repo.prestamos():
```

Justificación

- **Responsabilidades separadas:** el préstamo sabe *cuántos días* se atrasó cada ejemplar (si ya se devolvió, cuenta hasta la fecha real de devolución); la estrategia sabe *cuánto cuesta* ese atraso.
- **Abierto a extensión:** una política nueva es una clase nueva; Prestamo, los reportes y la web no cambian.
- **La estrategia se recibe como parámetro** en lugar de leerse adentro del préstamo: así `Prestamo` no depende de `Configuracion` (lo que además evita un import circular) y cada estrategia se puede probar con el mismo préstamo.

Consecuencias

- **Ventajas:** la política vigente se cambia en una sola línea de la configuración.
- **Desventajas:** quien calcula la multa debe pasar la estrategia; en el sistema siempre se toma de la configuración, de modo que el criterio es uniforme.

Verificación

`test_sin_atraso_no_hay_multa`, `test_estrategias_intercambiables` (el mismo préstamo da $1000, $600 o $0 según la estrategia), `test_ejemplar_devuelto_deja_de_acumular`, `test_estrategia_tomada_de_la_configuracion` y `test_reportes`.

6. Singleton — Configuración del sistema

Problema

Los plazos de préstamo (14 días el local y 21 el interbibliotecario) y la política de multas son parámetros de todo el sistema: los usan la fábrica de préstamos, los reportes y la interfaz web. Si cada módulo creara su propia configuración, un cambio en una no se vería en las otras, y un préstamo podría vencer con un plazo y cobrarse la multa con otra política.

Solución

`Configuracion` garantiza una única instancia con un único punto de acceso: cualquier llamada a `Configuracion()` devuelve el mismo objeto.

biblioteca/configuracion.py · líneas 9–30

```python
class Configuracion:
    _instancia = None
    _inicializada = False

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
        return cls._instancia

    def __init__(self):
        if Configuracion._inicializada:
            return
        self.plazo_prestamo_local_dias = 14
        self.plazo_prestamo_interbibliotecario_dias = 21
        self.estrategia_multa = MultaPorDia(100)
        Configuracion._inicializada = True

    @classmethod
    def reiniciar(cls):
        """Descarta la instancia actual (uso en tests)."""
        cls._instancia = None
        cls._inicializada = False
```

Justificación

- **Adaptación a Python:** la cátedra describe el patrón como «ocultar el constructor y crear un método estático que guarde la instancia». Python no tiene constructores privados; el equivalente es redefinir `__new__`, que es quien crea el objeto: la primera vez lo crea y lo guarda en `_instancia`, y las siguientes devuelve el guardado.
- **La bandera `_inicializada` es necesaria** porque Python ejecuta `__init__` en cada llamada a `Configuracion()`; sin ella, cada acceso volvería los valores a los de fábrica y se perderían los cambios.
- `reiniciar()` existe solo para los tests: cada test empieza con una configuración limpia (`setUp`/`tearDown`).
- **Dónde se decidió no aplicarlo:** la conexión a la base de datos *no* es un Singleton. Flask atiende cada petición en un hilo y una conexión sqlite3 no puede compartirse entre hilos, así que cada petición abre y cierra la suya.

Consecuencias

- **Ventajas:** consistencia garantizada y un único lugar para ajustar los valores de negocio.
- **Desventajas:** es estado global, lo que obliga a reiniciarlo entre tests. La creación no está protegida para hilos; en el peor caso dos hilos crearían instancias con los mismos valores por defecto, un riesgo aceptable para este sistema.

Verificación

`test_misma_instancia_y_estado_compartido`, `test_crea_prestamo_local_con_plazo_configurado` y `test_crea_prestamo_interbibliotecario_con_plazo_configurado`.

7. Factory — Creación de préstamos

Problema

Hay dos tipos de préstamo con reglas distintas. El **local** (misma sede) nunca genera remito y vence a los 14 días; el **interbibliotecario** exige sedes distintas, necesita un remito antes del despacho (RN11) y vence a los 21 días. Quien registra un préstamo (el formulario web) solo conoce el socio, la sede de origen, la de destino y los ejemplares. Si cada cliente decidiera qué clase instanciar y calculara el vencimiento, esa lógica se repetiría.

Solución

Rol del patrón	Clase	Responsabilidad
Producto abstracto	Prestamo (ABC)	Funcionalidad común: validaciones, detalles, vencimiento, multa. es_interbibliotecario() es abstracto.
Productos concretos	PrestamoLocal, PrestamoInterbibliotecario	Reglas propias de cada tipo; solo el interbibliotecario genera remitos.
Fábrica	FabricaPrestamos.crear_prestamo()	Elige la clase concreta y calcula el vencimiento con el plazo de la configuración.
Cliente	servicios.registrar_prestamo	Pide un préstamo y lo usa a través de la interfaz común.

Implementación

biblioteca/modelos/fabrica_prestamos.py · líneas 13–23

```
13  class FabricaPrestamos:
14      @staticmethod
15      def crear_prestamo(id_prestamo, socio, sede_origen, sede_destino, ejemplares, fecha_inicio=None):
16          config = Configuracion()
17          inicio = fecha_inicio or date.today()
18          if sede_origen is sede_destino:
19              vence = inicio + timedelta(days=config.plazo_prestamo_local_dias)
20              return PrestamoLocal(id_prestamo, socio, sede_origen, ejemplares, vence, inicio)
21          vence = inicio + timedelta(days=config.plazo_prestamo_interbibliotecario_dias)
22          return PrestamoInterbibliotecario(id_prestamo, socio, sede_origen, sede_destino,
23                                            ejemplares, vence, inicio)
```

El cliente no menciona ninguna subclase: trabaja con lo que devuelve la fábrica a través de la interfaz común de Prestamo:

biblioteca/servicios.py · líneas 92–103

```
92  def registrar_prestamo(repo, numero_socio, id_sede_origen, id_sede_destino, codigos, fecha_inicio=None):
93      _requerido(socio=numero_socio, sede_de_origen=id_sede_origen, sede_de_destino=id_sede_destino)
94      if not codigos:
95          raise ReglaNegocioError("Seleccione al menos un ejemplar.")
96      prestamo = FabricaPrestamos.crear_prestamo(
97          repo.siguiente_id_prestamo(), repo.socio(numero_socio), repo.sede(id_sede_origen),
98          repo.sede(id_sede_destino), [repo.ejemplar(c) for c in codigos], fecha_inicio)
99      if prestamo.es_interbibliotecario():
100         # El remito se genera al registrar el préstamo: así existe antes del despacho.
101         prestamo.generar_remito(repo.siguiente_numero_remito())
102     repo.guardar_prestamo(prestamo)
103     return prestamo
```

Justificación

- **El cliente no depende de las clases concretas**, que es el concepto clave del patrón según la cátedra: «encapsula la lógica de instanciación». Si mañana hubiera un tercer tipo de préstamo, cambiaría la fábrica, no los clientes.
- **Un solo criterio de decisión** (¿origen y destino son la misma sede?) y un solo lugar donde se calcula el vencimiento.
- **Combina dos patrones:** la fábrica toma los plazos del Singleton Configuracion.
- **Préstamos inválidos imposibles:** el constructor de Prestamo valida todas las reglas (socio habilitado, ejemplares disponibles y de la sede de origen, sedes activas) antes de crear el objeto.

Consecuencias y alcance

- **Variante aplicada:** se implementó una *fábrica simple* (un método de creación), no la jerarquía completa de fábricas abstractas y concretas de la presentación de la cátedra. Con dos productos y un único criterio, una jerarquía de fábricas agregaría clases sin beneficio. Si aparecieran más tipos de préstamo, se puede evolucionar a una fábrica abstracta con `FabricaPrestamoLocal` y `FabricaPrestamoInterbibliotecario` sin tocar a los clientes.
- **Desventaja:** el método de la fábrica cambia cada vez que se agrega un tipo de préstamo.

Verificación

`test_crea_prestamo_local_con_plazo_configurado`, `test_crea_prestamo_interbibliotecario_con_plazo_configurado`, `test_interbibliotecario_exige_sedes_distintas` y `test_prestamo_local_no_genera_remito`.

8. Observer — Notificaciones al socio

Problema

Entre los procesos opcionales está el envío de notificaciones: el socio quiere enterarse cuando su material se empaqueta, se despacha y llega. Si `Remito` llamara directamente a un notificador, el dominio quedaría atado a un canal concreto (mail, pantalla, base de datos), y agregar otro canal obligaría a modificar el remito.

Solución

Rol del patrón	Clase	Responsabilidad
Sujeto	Sujeto (observador.py)	Mantiene la lista de observadores: agregar, quitar y notificar.
Sujeto concreto	Remito	Hereda de Sujeto y notifica en cada cambio de estado.
Observador (interfaz)	Observador	Declara actualizar(sujeto).
Observador concreto	NotificadorSocio	Arma el mensaje para el socio del préstamo.
Suscripción y canal	servicios.avanzar_remito, Repositorio	Suscribe el notificador, ejecuta la operación y guarda los avisos en la tabla notificacion (página «Notificaciones» de la web).

Implementación

biblioteca/observador.py · líneas 6–25

```python
class Observador(ABC):
    @abstractmethod
    def actualizar(self, sujeto):
        pass


class Sujeto:
    def __init__(self):
        self._observadores = []

    def agregar_observador(self, observador):
        if observador not in self._observadores:
            self._observadores.append(observador)

    def quitar_observador(self, observador):
        self._observadores.remove(observador)

    def notificar(self):
        for observador in self._observadores:
            observador.actualizar(self)
```

biblioteca/notificaciones.py · líneas 4–19

```python
class NotificadorSocio(Observador):
    """Observa remitos y genera un aviso al socio en cada cambio de estado.

    Por ahora guarda los mensajes en una lista; el canal real (mail, pantalla)
    se define cuando exista la interfaz.
    """

    def __init__(self):
        self.notificaciones = []

    def actualizar(self, remito):
        ultimo = remito.historial[-1]
        socio = remito.prestamo.socio
        mensaje = (f"Remito {remito.numero_remito}: su préstamo {remito.prestamo.id_prestamo} "
                   f"pasó a estado {ultimo.nombre_estado}.")
        self.notificaciones.append((socio, mensaje))
```

Punto de unión con State: la notificación se dispara dentro de `cambiar_estado()`, que solo se ejecuta cuando una transición fue válida:

biblioteca/modelos/remito.py · líneas 67–70

```python
    def cambiar_estado(self, nuevo_estado, sede_actual, observacion=""):
        self._estado = nuevo_estado
        self.registrar_trazabilidad(nuevo_estado.nombre, sede_actual, observacion)
        self.notificar()
```

biblioteca/servicios.py · líneas 106–117

```python
def avanzar_remito(repo, numero_remito, operacion, observacion=""):
    """operacion: 'empaquetar', 'despachar' o 'recibir' (el estado actual decide si se admite)."""
    if operacion not in ("empaquetar", "despachar", "recibir"):
        raise ReglaNegocioError(f"Operación desconocida: {operacion}.")
    remito = repo.remito(numero_remito)
    notificador = NotificadorSocio()
    remito.agregar_observador(notificador)
    getattr(remito, operacion)(observacion.strip())
    remito.quitar_observador(notificador)
    repo.guardar_prestamo(remito.prestamo)
    repo.guardar_notificaciones(notificador)
    return remito
```

Justificación

- **Desacoplamiento uno a muchos:** el remito no sabe quién lo observa ni qué hace con el aviso. Un canal nuevo (por ejemplo, un mail real) es otra clase Observador, sin modificar el remito.
- **Se notifica exactamente lo que ocurrió:** como la notificación está en el único punto por el que pasan las transiciones válidas, los intentos rechazados por el State no generan avisos.
- **El dominio no conoce la base de datos:** el observador acumula los mensajes y es el caso de uso el que los guarda con el Repositorio.

Consecuencias

- **Ventajas:** se pueden sumar o quitar canales de aviso en tiempo de ejecución.
- **Desventajas:** las suscripciones son objetos en memoria y no se guardan en la base, por eso el caso de uso suscribe el observador en cada operación. Los observadores se notifican en el orden en que se suscribieron.

Verificación

`test_socio_notificado_en_cada_cambio_de_estado`, `test_observador_quitado_no_recibe_mas`, `test_transicion_rechazada_no_notifica` y `test_ciclo_completo_con_retorno_y_notificaciones` (seis avisos: tres de la ida y tres del retorno).

9. Cómo colaboran los patrones en un caso de uso

Los patrones no están aislados. Este es el recorrido de un préstamo interbibliotecario completo, tal como se ejecuta desde la interfaz web:

Paso	Qué ocurre	Patrones
1. Consultar disponibilidad	El formulario lista los ejemplares disponibles de la sede de origen.	Iterator
2. Registrar préstamo	La fábrica elige PrestamoInterbibliotecario con vencimiento a 21 días; el constructor valida las reglas; se genera el remito en estado Pendiente con su primera entrada de historial.	Factory, Singleton, State
3. Empaquetar, despachar, recibir	El caso de uso suscribe el NotificadorSocio; el estado actual decide si la operación es válida y ejecuta sus efectos; cada transición notifica al socio.	State, Observer
4. Devolución	Solo se admite si el remito de ida está Entregado; el ejemplar sigue no disponible. Al devolver todo se genera el remito de retorno.	State
5. Retorno	El remito de retorno recorre los mismos estados; al recibirse, los ejemplares quedan disponibles en su sede y vuelven a aparecer en el paso 1.	State, Observer, Iterator
6. Reportes	Las multas de préstamos vencidos se calculan con la estrategia configurada.	Strategy, Singleton

10. Técnicas complementarias

No son patrones exigidos por la cátedra, pero sostienen a los anteriores al guardar los datos en SQLite:

- **Repositorio** (`biblioteca/persistencia/repositorio.py`): concentra todo el SQL. Ni el dominio ni la web escriben consultas para leer o guardar objetos.
- **Mapa de identidad:** dentro de una petición, cada fila se convierte en un único objeto. Es necesario porque el dominio compara sedes por identidad (`is`), por ejemplo en `Ejemplar.pertenece_a_sede()`.
- **Reconstrucción del estado (State persistido):** en la base se guarda el nombre del estado; al leer, un diccionario lo traduce a la clase de estado correspondiente:

biblioteca/persistencia/repositorio.py · líneas 19–21

```
19  ESTADOS_REMITO = {c.nombre: c for c in (Pendiente, Empaquetado, Enviado, Entregado)}
20  ESTADOS_RESERVA = {c.nombre: c for c in (ReservaPendiente, ReservaConfirmada,
21                                           ReservaCumplida, ReservaCancelada)}
```

biblioteca/persistencia/repositorio.py · líneas 261–266

```
261          r = Remito.__new__(Remito)
262          Sujeto.__init__(r)
263          r.numero_remito = f["numero_remito"]
264          r.prestamo = prestamo
265          r.tipo = f["tipo"]
266          r._estado = ESTADOS_REMITO[f["estado"]]()
```

Además, las restricciones de la base (claves primarias, UNIQUE y un índice único parcial que impide que un ejemplar tenga dos préstamos pendientes) funcionan como última barrera para las reglas de no duplicación, por si alguna validación se saltea por error.